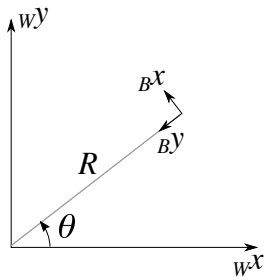
Prof. Dr. Stefan Leutenegger
Teaching Assistants: Leonard Freissmuth and Leonard Diederich

Solution E1: State and Map Representations

Session date: 24/02/2026

Task 1: 2D Lawn-mower Robot

Consider a lawn-mower robot (robot frame $\underrightarrow{\mathcal{F}}_B$) moving around in the $\underrightarrow{\mathcal{F}}_W$ x-y plane in a way parameterised by time as follows:



$$
\begin{aligned}
{}_W x &= R\cos(\omega t),\\
{}_W y &= R\sin(\omega t),\\
\theta &= \omega t,
\end{aligned}
$$

where θ denotes the robot orientation as indicated in the sketch.

i) Write down the homogeneous transformation matrix $\boldsymbol{T}_{WB}$ (for the full 6D pose) as a function of time.

Note that the robot's x-axis is pointing into y direction for $t = 0$. Therefore, it is easiest to break down the rotation as $\mathbf{R}_{WB}(t) = \mathbf{R}_{WB_0}\mathbf{R}_{B_0B}(t)$. To find $\mathbf{R}_{WB_0}$ we observe that it is obtained with a rotation around the z-axis by 90 degrees, i.e. we find

$$
\mathbf{R}_{WB_0} = \begin{bmatrix} \cos\frac{\pi}{2} & -\sin\frac{\pi}{2} & 0 \\ \sin\frac{\pi}{2} & \cos\frac{\pi}{2} & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}.
$$

For the time-dependent part, we find an additional rotation of $\theta = \omega t$ around the z-axis, i.e.

$$
\mathbf{R}_{B_0B}(t) = \begin{bmatrix} \cos\omega t & -\sin\omega t & 0 \\ \sin\omega t & \cos\omega t & 0 \\ 0 & 0 & 1 \end{bmatrix}.
$$

As for the position, we find that

$$
{}_W\mathbf{t}_B(t) = \begin{bmatrix} R\cos(\omega t) \\ R\sin(\omega t) \\ 0 \end{bmatrix},
$$

where the z-component was assumed to be zero for the robot position to stay in the x-y plane. Now we can assemble the overall homogeneous transformation matrix as

$$
\boldsymbol{T}_{WB}(t) = \begin{bmatrix} \mathbf{R}_{WB}(t) & {}_W\mathbf{t}_B(t) \\ 0 & 1 \end{bmatrix}.
$$

ii) As alternative forms of representing the orientation, compute the axis-angle and quaternion $\mathbf{q}_{WB}$ as functions of time.

> Again, we can use the same composition as above. In terms of axis and angle, we will find that the initial rotation $\mathbf{R}_{WB_0}$ corresponds to a normal $\mathbf{n}_{WB_0} = [0, 0, 1]^T$ and an angle $\alpha_{WB_0} = \pi/s$. The second rotation then similarly follows with $\mathbf{n}_{B_0B} = [0, 0, 1]^T$ and $\alpha_{B_0B} = \omega t$. We can compute the respective quaternions by plugging them into the formula $q = [\mathbf{n} \sin \alpha/2, \cos \alpha/2]^T$.
> With this, we can now compose the overall rotation as $\mathbf{q}_{WB} = \mathbf{q}_{WB_0} \otimes \mathbf{q}_{B_0B}$. Note how this composition is not trivial to do directly in angle-axis representation. However, we can easily convert the quaternion $\mathbf{q}_{WB}$ back to angle-axis.

iii) Now assume that the robot actually is equipped with a magnetometer that measures the magnetic field vector. The magnetic field vector is assumed to be known in the world frame for the specific location where the lawn-mower is deployed, and amounts to ${}_W\mathbf{m}$. Derive a measurement model $\mathbf{z} = \mathbf{h}(\mathbf{x}) + \mathbf{v}$, where $\mathbf{x} = [{}_Wx, {}_W y, \theta]^T$ and $\mathbf{v}$ denotes noise.

> Note that the magnetometer will measure in its sensor frame. We can assume that it is mounted in $\underrightarrow{\mathcal{F}}_B$ (if it were not, we could add an additional, known, static transform $\boldsymbol{T}_{BS}$). Now, the measurement can be written as $\mathbf{z} = \mathbf{R}_{BW}({}_Wx, {}_W y, \theta){}_W\mathbf{m} + \mathbf{v}$, where $\mathbf{R}_{BW}({}_Wx, {}_W y, \theta) = \mathbf{R}_{WB}{}^T$ as derived above.

Task 2: Visualising an Aeroplane (Python)

Open the Python3 script `e01_aeroplane.py` in a (virtual) python environment where you have the package `vedo==2025.5.4` installed. There, you will find a visualiser with sliders to set the Tait-Bryan angles ϕ, θ, ψ (roll, pitch, yaw) that describe the orientation of an aeroplane (frame $\underrightarrow{\mathcal{F}}_B$) relative to world frame $\underrightarrow{\mathcal{F}}_E$. The world frame is already being drawn. Now it is your task to finish writing this visualiser. You should follow the convention regarding aeronautics from the lecture, i.e. the world-frame (Earth-fixed, tangential) $\underrightarrow{\mathcal{F}}_E$ should be North-East-Down (NED).

> For a reference implementation of the task below, see `e01_aeroplane_sol.py`.

i) Familiarise yourself with the code and get the Jupyter notebook to run by installing the necessary dependencies (notably `vedo`). It should interactively visualise the aeroplane in 3D with zero roll, pitch, and yaw, and the sliders won't yet have an effect on setting orientation.

ii) The visualiser should now put the aeroplane (frame $\underrightarrow{\mathcal{F}}_B$) into the right orientation, and also plot its coloured x-y-z axes. Note how the aeroplane mesh was defined in a z-up convention, making an additional transform necessary to flip it around in the end (which is already implemented).

iii) Now, in addition, there are two cameras (index $i \in 1, 2$) mounted at the wingtips, positioned ${}_B\mathbf{t}_{C_i} = [0, \pm 2, 0]$. They are facing forward (i.e. the camera z-axes are pointing along the $\underrightarrow{\mathcal{F}}_B$ x axis) and the camera x-axes point along the y axis of $\underrightarrow{\mathcal{F}}_B$. Derive the full transformations $\boldsymbol{T}_{BC_i}$ and plot the respective camera frame axes into your interactive visualisation, too.

Task 3: Probability Density Functions

Consider the following Gaussian distribution for two continuous random variables $\mathbf{x} = [x_1, x_2]^T$:

$$p(\mathbf{x}) = \frac{1}{2\pi\sqrt{\det \boldsymbol{\Sigma}}} \exp\left(-\frac{1}{2}\left((\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})\right)\right),$$

where the means are $\boldsymbol{\mu} = [\mu_1, \mu_2]^T$ and $\boldsymbol{\Sigma}$ denotes the 2-by-2 covariance matrix.

i) Prove that for $\boldsymbol{\Sigma} = \mathrm{diag}[\sigma_1^2, \sigma_2^2]$ with $\sigma_{1,2} \in \mathbb{R}$, x_1 and x_2 are independent. Also compute the respective distributions $p(x_1)$ and $p(x_2)$, further pointing out their means and standard deviations.

We can plug in $\boldsymbol{\Sigma} = \mathrm{diag}[\sigma_1^2, \sigma_2^2]$ above and get

$$\begin{aligned} p(\mathbf{x}) = & \quad \frac{1}{2\pi\sqrt{\sigma_1^2\sigma_2^2}} \exp\left(-\frac{1}{2}\left((\mathbf{x}-\boldsymbol{\mu})^T \mathrm{diag}\left[\frac{1}{\sigma_1^2}, \frac{1}{\sigma_2^2}\right](\mathbf{x}-\boldsymbol{\mu})\right)\right) \\ = & \quad \frac{1}{2\pi\sqrt{\sigma_1^2\sigma_2^2}} \exp\left(-\frac{1}{2}\left(\frac{(x_1-\mu_1)^2}{\sigma_1^2} + \frac{(x_2-\mu_2)^2}{\sigma_2^2}\right)\right), \end{aligned}$$

which was obtained by multiplying out the square form in the exponential function. Now with basic rules of exponentiation, we can write this as two factors

$$\begin{aligned} p(\mathbf{x}) = & \quad \frac{1}{2\pi\sqrt{\sigma_1^2\sigma_2^2}} \exp\left(-\frac{1}{2}\left(\frac{(x_1-\mu_1)^2}{\sigma_1^2}\right)\right) \exp\left(-\frac{1}{2}\left(\frac{(x_2-\mu_2)^2}{\sigma_2^2}\right)\right) \\ = & \quad \frac{1}{\sqrt{2\pi}\sigma_1}\frac{1}{\sqrt{2\pi}\sigma_2} \exp\left(-\frac{1}{2}\left(\frac{(x_1-\mu_1)^2}{\sigma_1^2}\right)\right) \exp\left(-\frac{1}{2}\left(\frac{(x_2-\mu_2)^2}{\sigma_2^2}\right)\right), \end{aligned}$$

corresponding to the product of two Normal distributions $x_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $x_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$.

ii) Now compute $p(x_1|x_2 = 0)$ for an arbitrary (positive semi-definite) covariance matrix $\boldsymbol{\Sigma} = \begin{bmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{12} & \Sigma_{2,2} \end{bmatrix}$. What are mean and standard deviation?

First, let us introduce

$$\boldsymbol{\Sigma}^{-1} = \begin{bmatrix} w_{11} & w_{12} \\ w_{12} & w_{22} \end{bmatrix},$$

which we could (analytically) compute relatively easily. Next, let's remember that we have $p(x_1, x_2) = p(x_1|x_2)p(x_2)$. Thus we can write $p(x_1|x_2) = kp(x_1, x_2)$, since $p(x_2)$ will simply be a scalar constant that we can ultimately compute so the distribution integrates to 1. Also, the factor before the exponent is in the following just written as a constant c for simplicity. Now plugging in $x_2 = 0$ and multiplying out the product in the exponential, we obtain

$$\begin{aligned} & p(x_1|x_2 = 0) = kp(x_1, 0) \\ & = kc \exp\left(-\frac{1}{2}\left(((x_1-\mu_1)w_{11} - \mu_2 w_{12})(x_1-\mu_1) - \mu_2((x_1-\mu_1)w_{12} - \mu_2 w_{22})\right)\right) \\ & = kc \exp\left(-\frac{1}{2}\left((x_1-\mu_1)^2 w_{11} + (x_1-\mu_1)(-2\mu_2 w_{12}) + \mu_2^2 w_{22}\right)\right) \\ & = kc \exp\left(-\frac{1}{2}\left(x_1^2 w_{11} - (2w_{11}\mu_1 + 2\mu_2 w_{12})x_1 + (2\mu_1\mu_2 w_{12} + \mu_1^2 w_{11} + \mu_2^2 w_{22})\right)\right). \end{aligned}$$

We observe that the exponent looks similar to the one of a $\mathcal{N}(\mu, \sigma^2)$, namely

$$-\frac{1}{2}\left(\frac{(x_1-\mu)^2}{\sigma^2}\right) = -\frac{1}{2}\left(\frac{1}{\sigma^2}(x_1^2 - 2\mu x_1 + \mu^2)\right).$$

We find that for $\sigma^2 = \frac{1}{w_{11}}$ and $\mu = (\mu_1 + \mu_2 w_{12}/w_{11})$, they will be equal – except for the constant part:

$$2\mu_1\mu_2 w_{12} + \mu_1^2 w_{11} + \mu_2^2 w_{22} \neq (\mu_1 + \mu_2 w_{12}/w_{11})^2.$$

Note, however, that adding a constant inside the exponential function is equivalent to multiplication of the whole expression with a constant, and therefore we can ignore this inequality.

Task 4: Balloons in Cappadocia (Blender)

Figure 1: Balloons in Cappadocia, Image by Robert_K on Pixabay: `https://pixabay.com/photos/cappadocia-hot-air-balloons-turkey-6771879/`

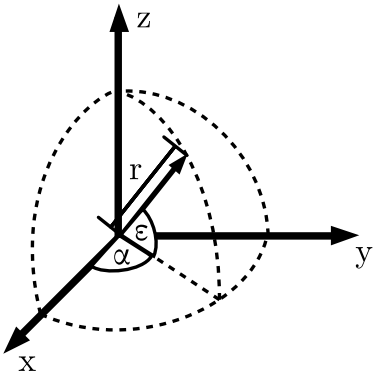


Figure 2: Polar coordinates used in this exercise.

You are on a hot-air balloon in Cappadocia and want to film your friend, who skydives from a second hot-air ballon. You use a camera mounted on a gimbal rigidly attached to the balloon basket. Both balloons translate and rotate during the flight.

Let $\underrightarrow{\mathcal{F}}_W$ denote the world frame. The body frames of the balloons are $\underrightarrow{\mathcal{F}}_{B_1}$ (filming balloon) and $\underrightarrow{\mathcal{F}}_{B_2}$ (jump balloon). Their poses with respect to the world frame are provided by GNSS and magnetometer measurements as homogeneous transforms:

$$\boldsymbol{T}_{WB_i} \in SE(3), \quad i \in \{1, 2\}.$$

The skydiver cannot carry a GNSS and streaming streaming setup, but instead wears a passive radar reflector. Balloon 2 carries a very accurate and long-range radar that measures the relative position of the skydiver in polar coordinates expressed in $\underrightarrow{\mathcal{F}}_{B_2}$ (See Fig. 2)

The camera is rigidly mounted on balloon 1 at a fixed translation

$${}_{B_1}\mathbf{t}_{B_1C} = [0, -4, -19]^{\mathrm{T}},$$

with camera frame $\underrightarrow{\mathcal{F}}_C$ defined as follows:

- $z_\mathcal{C}$ points along the viewing direction,
- $x_\mathcal{C}$ points to the right in the image,
- $y_\mathcal{C}$ points down in the image.

In this exercise, you will use Blender, which can be downloaded from https://blender.org. You will receive a Blender scene file, which has been tested on version 4.5.7 LTS and contains:

- two objects representing the balloon frames $\underrightarrow{\mathcal{F}}_{B_1}$ and $\underrightarrow{\mathcal{F}}_{B_2}$,
- an animated skydiver object,

- an animated camera that zooms towards the skydiver,
- an object representing the camera pose $\boldsymbol{T}_{WC}$ to be updated each frame.

To see the console output of your blender script, you need to start blender from the terminal if you are on Linux. In windows, you can show the console output through the menu ribbon and going to Window → Toggle System Console. The viewports show a 3D view on the top left, the camera view on the bootom left and a code editor on the right. In Blender, use:

- **Middle mouse drag** to orbit the view (or click and drag the 3D gizmo on the top right of the 3D view port if you are on a laptop),
- **Shift + middle mouse drag** to pan (or use the hand icon below the gizmo if you are on a laptop),
- **Scroll wheel** to zoom (or a trackpad scroll motion if you are on a laptop),
- **Space bar** (while hovering the mouse over the 3D view port) to play and pause the animation.

The exercise is implemented as a Python *frame change handler* that runs once per animation frame. Your task is to complete the computations so that the camera continuously points at the skydiver throughout the animation. To register the frame change handler, execute the script with the triangle button at the top ribbon of the text editor. When you then play back the animation with the spacebar, you'll see your code in action.

> For a reference implementation of the task, see `balloons_in_cappadocia_solution.blend`.

1. Convert the radar polar measurement (r, α, ε) to Cartesian translation in $\underrightarrow{\mathcal{F}}_{B_2}$.

$$ {}_{B_2}\mathbf{t}_{B_2S} = [{}_{B_2}x(r,\alpha,\varepsilon), {}_{B_2}y(r,\alpha,\varepsilon), {}_{B_2}z(r,\alpha,\varepsilon)]^{\mathrm{T}} $$

2. Compute the camera rotation with respect to $\underrightarrow{\mathcal{F}}_{B_1}$ such that it points at the skydiver. Using the provided $\boldsymbol{T}_{WB_1}$, $\boldsymbol{T}_{WB_2}$, and ${}_{B_1}\mathbf{t}_{B_1C}$ as well as the radar-derived ${}_{B_2}\mathbf{t}_{B_2S}$, compute the skydiver position expressed in $\underrightarrow{\mathcal{F}}_{C}$. Afterwards, compute the camera's rotation matrix $\mathbf{R}_{B_1C}$ such that:
   - The z-axis of $\underrightarrow{\mathcal{F}}_{C}$, i.e. the viewing direction, aligns with ${}_{C}\mathbf{t}_{CS}$,
   - The x-axis of $\underrightarrow{\mathcal{F}}_{C}$ aligns with the local horizon, i.e. lies in xy-plane of $\underrightarrow{\mathcal{F}}_{W}$.

   (Handle the degenerate case where the viewing direction is parallel to "down".)
3. Update $\boldsymbol{T}_{WC}$ by assembling $\mathbf{R}_{B_1C}$ and ${}_{B_1}\mathbf{t}_{B_1C}$ into a homogeneous transform and projecting it into world frame.